RDBMS & Database Programming With JDBC

Total Time: 3 Hours

Theme: UPI-Style Peer-to-Peer Wallet

Section A: Conceptual Understanding

1. Explain how Java runtime enables this wallet to work across operating systems.
2. Explain precision handling for money transfers.
3. Explain transfer validation logic.
4. Explain loop usage for monthly summaries.
5. Explain why collections suit transaction storage.
6. Explain payment modes using polymorphism.

Section B: Practical Tasks

1. **User with Encapsulated Balance:** Implement a `User` class with a `private double balance`. Use `public` getter and setter methods to control access, ensuring the balance is never modified directly.
2. **Constructor Overloading:** Define multiple constructors for the `User` class: one for a basic profile (name, UPI ID) and another for a premium profile that includes an initial account balance.
3. **PaymentMethod Interface:** Create a `PaymentMethod` interface with a `processPayment(double amount)` method. Implement this in `BankTransfer` and `WalletBalance` classes to provide different transaction options.
4. **InsufficientBalanceException:** Develop a custom exception class `InsufficientBalanceException`. Write logic to `throw` this exception if a transaction amount exceeds the current encapsulated balance.
5. **Mask UPI ID:** Implement a String manipulation method using `substring()` and `charAt()` to mask the middle of a UPI ID (e.g., `user@upi` becomes `u***@upi`) for data privacy.

Section C: Mini Project

Objective: Build a robust console-based payment application that manages user balances and financial logs using **Java and MySQL/Postgres** via JDBC.

Requirements & Tasks:

1. **Identity Management:** Implement **Register and Login** functionality.
    - Authenticate users against a `users` table.
    - Use **PreparedStatements** to prevent SQL injection during credential verification.
2. **Wallet Operations (Add/Send Money):** Create the core transaction logic.
    - **Send Money:** Implement **SQL Transactions** (`setAutoCommit(false)`) to ensure that deducting money from one account and adding it to another is an "all-or-nothing" operation.
    - Handle insufficient balance scenarios with custom Java exceptions.
3. **Transaction History:** Maintain a permanent ledger in a `transactions` table. Users should be able to view their recent activity, including sender details, receiver details, timestamps, and status.
4. **File Persistence (Export):** Add a feature to **Export Statement to File**. Generate a `.txt` or `.csv` summary of the user's transaction history using Java I/O (FileWriter/BufferedWriter).